Prova di Informatica@DSS (2026/2027)

Donatella Firmani Paolo Franciosa
Massimo Lauria

None

Modalità di lavoro: Per l'esame potete avere al tavolo solo

- penne, matite, cancelleria;
- un documento;
- una bevanda e uno snack;

e dovete lasciare borse e zaini in un angolo della sala, con cellulari e dispositivi elettronici spenti al loro interno. Noi forniremo i fogli.

Il testo della prova, i file di test degli esercizi e una tavola sinottica sulle sintassi Python si trovano nella cartella `esame` sulla scrivania, nel sistema "Statistica-Laboratorio". Le soluzioni degli esercizi devono essere messe nella stessa cartella `esame`. **Per prima cosa compilate il file** `studente.txt` con le informazioni di nome, cognome e matricola, secondo la struttura

```
nome: Giacomo
cognome: Leopardi
matricola: 123456789
```

Non compilare il file `studente.txt` è come non consegnare il compito.

Il voto è la somma dei punti degli esercizi, pertanto è possibile avere 30 anche senza completarli tutti.

Uso dei file di test: i test di questa simulazione controllano molto sommariamente l'impostazione delle soluzioni e passare i test non implica affatto che la soluzione sia corretta. Leggete bene il testo degli esercizi prima di iniziare a risolverli. Per portare a termine ogni esercizio è necessario

- scrivere un file di soluzione **col nome specificato**;
- avere dentro la funzione **col nome specificato**;
- porre il file di soluzione nella cartella `esame`;
- eseguire in quella cartella il comando `python3 <fileditest>`

dove `<fileditest>` va ovviamente sostituito con il nome del file di test appropriato per l'esercizio su cui state lavorando.

Ogni esercizio di programmazione ha un file di test corrispondente. Il risultato di ogni test mostra:

- se è stato trovato il file con il nome corretto,
- se il file contiene la funzione con il nome corretto,
- se chiamate alla funzione con diversi valori dei parametri terminano restituendo risultati corretti.

Per ogni funzione scritta vengono eseguite chiamate con diversi valori dei parametri. L'esito dei test viene riportato con il carattere

- `E` se la chiamata non può essere eseguita,
- `F` se la funzione non restituisce il risultato corretto,
- `.` se la funzione restituisce il risultato corretto.

1 Somma di liste

Scrivere una funzione `sommaliste(a, b)` che:

- si aspetti come argomenti due liste `a`, `b` della stessa lunghezza, in cui ciascun elemento è un numero `int`;
- crei e restituisca una lista, della stessa lunghezza di quelle ricevute come argomenti, in cui l'elemento i-esimo della lista è uguale alla somma dell'elemento i-esimo di `a` e l'elemento i-esimo di `b`.

Ad esempio la somma delle liste `[3, 5, 1]` e `[1, -2, 4]` è `[4, 3, 5]`.

Il vosto programma può **assumere** che le due liste in input abbiano la stessa lunghezza e siano costituite da numeri interi

Il programma Python deve essere salvato nel file: `sommaliste.py`

File di test: `test_sommaliste.py`

2 Prodotto Scalare tra due vettori

Il prodotto scalare tra due vettori $\langle a_0, a_1, \ldots, a_{n-1} \rangle$ e $\langle b_0, b_1, \ldots, b_{n-1} \rangle$ è il valore

$$\sum_{i=0}^{n-1} a_i b_i$$

Le liste sono ovviamente un modo naturale per rappresentare il concetto matematico di vettore. Scrivere una funzione `prodottoscalare(a, b)` che:

- riceva come parametri due liste non vuote `a`, `b` della stessa lunghezza, in cui ciascun elemento è un numero (`int` oppure `float`);
- restituisca il prodotto scalare delle due liste. Il prodotto scalare tra due liste è quindi la somma dei prodotti degli elementi omologhi. Ad esempio, il prodotto scalare tra i vettori `[1, 4, 3]` e `[5, 5, 1]` è $1 \times 5 + 4 \times 5 + 3 \times 1 = 28$. Il prodotto scalare di due vettori è un valore numerico. Osservate che il prodotto scalare di due liste vuote è 0.

Potete **assumere** che le liste non siano vuote, che contengano solo numeri e che abbiano la stessa lunghezza.

Il programma Python deve essere salvato nel file: `prodottoscalare.py`

File di test: `test_prodottoscalare.py`

3 Separa elementi in base alla media

Scrivere una funzione `separaelementi(valori)` che:

- si aspetti come argomento una lista `valori`, in cui ciascun elemento è un numero (`int` oppure `float`);
- restituisca due liste (notare che è ammessa l'istruzione `return lista1, lista2` per restituire due liste), dove `lista1` deve contenere tutti gli elementi di valori che sono maggiori della media degli elementi in `valori`, mentre `lista2` deve contenere tutti gli elementi che sono minori o uguali alla media degli elementi in `valori`.

Ad esempio, se la lista fosse `[0.96, 0.2, 0.4, 0.1, 0.55, 0.03, 0.88]`, poiché la media degli elementi è `0.4457`, si dovrebbero restituire le liste `[0.96, 0.55, 0.88]` e `[0.2, 0.4, 0.1, 0.03]`.

All'interno di ognuna delle due liste gli elementi devono comparire nello stesso ordine in cui compaiono nella lista originale.

Il programma Python deve essere salvato nel file: `separaelementi.py`

File di test: `test_separaelementi.py`

4 Intersezione

Scrivere una funzione `intersezione(a, b)` che:

- si aspetti come argomenti due liste `a`, `b`;
- restituisca una lista **ordinata** che rappresenti l'intersezione tra le due liste, cioè gli elementi che appartengono sia alla lista `a` che alla lista `b`. Si può assumere che `a` e `b` non contengano rispettivamente doppioni.

Ad esempio

```
L = intersezione([2,4,3,1],[4,1,5])
print(L)
```

```
[1, 4]
```

Il programma Python deve essere salvato nel file: `intersezione.py`

File di test: `test_intersezione.py`

5 Unione

Scrivere una funzione `unione(a, b)` che si aspetti come argomenti due liste `a`, `b` e restituisca una lista **ordinata** che rappresenti l'unione tra le due liste `a` e `b`, cioè gli elementi che appartengono alla lista `a` e/o alla lista `b`.

Si può assumere che `a` e `b` non contengano rispettivamente doppioni. Tuttavia un elemento potrebbe apparire sia in `a` che in `b`, e deve apparire una volta sola nella loro unione. Per ordinare la risposta è sufficiente usare la funzione `sorted` o il metodo `sort`.

Il programma Python deve essere salvato nel file: `unione.py`

File di test: `test_unione.py`